❶ Indicates a class as a source of Spring configuration

❷ Enables R2DBC auditing for persistent entities

Having defined the data you want to persist, you can go ahead and explore how to access it.

USING REACTIVE REPOSITORIES

Spring Data provides the *repository* abstraction for all the modules in the project, including R2DBC. The only difference between this and what you did in chapter 5 is that you're going to use a reactive repository.

In the `com.polarbookshop.orderservice.order.domain` package, create a new `OrderRepository` interface and make it extend `ReactiveCrudRepository`, specifying the type of data handled (`Order`) and the data type of the `@Id`-annotated field (`Long`).

Listing 8.8 Repository interface for accessing orders

```
package com.polarbookshop.orderservice.order.domain;

import org.springframework.data.repository.reactive.ReactiveCrudRepository;

public interface OrderRepository
  extends ReactiveCrudRepository<Order,Long> {}    ❶
```

❶ Extends a reactive repository providing CRUD operations, specifies the type of managed entity (Order), and its primary key type (Long)

The CRUD operations provided by `ReactiveCrudRepository` are enough for the use cases of the Order Service application, so you won't need to add any custom methods. However, we're still missing the `orders` table in the database. Let's define it using Flyway.